

Projektbeskrivning

Gameboy Emulator

2025-03-25

Projektmedlemmar:

Natalie Holmström <natho280@student.liu.se>

Fritiof Santesson <frisa670@student.liu.se>

Handledare:

Stina Åström <stias606@student.liu.se>

Innehåll

1. Introduktion till projektet.....	2
2. Ytterligare bakgrundsinformation.....	2
3. Milstolpar.....	2
4. Övriga implementationsförberedelser.....	4
5. Utveckling och samarbete.....	4
6. Implementationsbeskrivning.....	6
6.1. Milstolpar.....	6
6.2. Dokumentation för programstruktur, med UML-diagram.....	6
7. Användarmanual.....	7

Projektplan

1. Introduktion till projektet

Vi ska göra en gameboy-emulator som ska kunna läsa in en binär fil (ROM-fil) innehållande ett Game Boy-spel och låta användaren spela det.

I grova drag består en emulator av en modell av den riktiga hårdvaran, avläsning av binära instruktioner från fil, och tolkning av de binära instruktionerna för att manipulera hårdvaran. Till sist presenteras displayen och knappar att trycka på till användaren – precis som på den riktiga konsolen.

Vi kommer också sikta på att implementera debugging-tillägg som möjligheten att inspektera minnet, och eventuellt dekompilering (egentligen bara en lista på de instruktioner som finns i programmet). Vi lämnar utrymme för att det sistnämnda kan vara mycket svårare än vi förutser, men vi vill i alla fall ha det i åtanke. Detta för att inte snöa in helt på att skriva själva emulatorn, vilket kan vara svårt att hålla “rent objektorienterat”, och vill därför ha några extra features som lämpar sig bättre för att visa upp våra kunskaper i Java.

2. Ytterligare bakgrundsinformation

För att skriva en emulator krävs mycket information om hårdvarans funktionalitet, och även hur instruktionerna ser ut och fungerar. I teorin kan man hitta allt detta genom reverse engineering (demontering), men med tanke på all information som går att hitta online bortser vi från detta då vi anser det som överkomplicerat och även onödigt. Vi använder oss (i alla fall inledningsvis) av följande webbsidor, som kompletteras av diverse mer specifika google-sökningar o.d.

<https://meganesu.github.io/generate-gb-opcodes/>

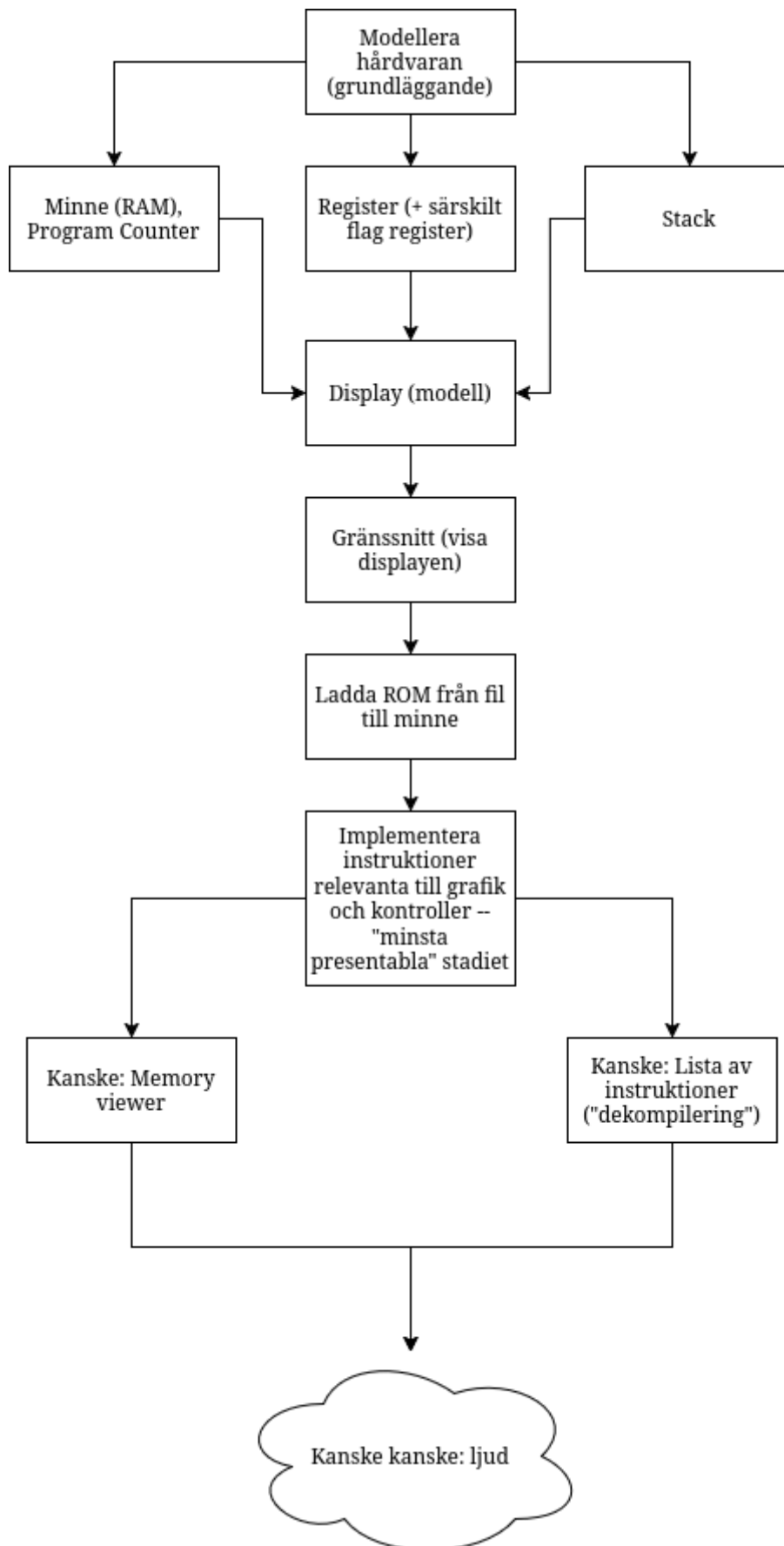
<https://gbdev.io/pandocs/>

Länkarna ovanför ger en överblick av hårdvaran och instruktionerna vi behöver implementera.

Sist men inte minst kommer vi mest troligt att ignorera vissa egenskaper hos gameboy-hårdvaran fullständigt: det finns en IR-port, länkkabel, och självklart ljud och musik. De två första är ingenting som krävs för att kunna spela 99% av alla spelen, så vi siktar inte ens på att genomföra dem.

Ljud är något som vi klarar oss utan för att kunna visa upp projektet, och är lätt att “strunta i” medan man implementerar emulatorn. Så, om vi inte skulle finna oss med massor och åter massor av extra tid, är det inget vi kommer försöka oss på.

3. Milstolpar



#	Beskrivning
1	Modell av hårdvara (flera steg):
2	RAM: val av datatyp och struktur
3	RAM: Hur många bitar på varje plats?
4	RAM: Hur många bitar i adressen?
5	RAM: Fundera över lämpliga metoder vi vet med oss kommer behövas (detta KOMMER vi behöva återgå till senare)
6	Register: hur många, vilken datatyp?
7	Register: Metoder, setter/getters, hur ska vi “snyggt” komma åt enskilda register?
8	Stack: Vart ligger den, hur fungerar den – ska man kunna spara godtyckliga värden eller bara värdet av PC vid hopp? Implementera.
9	Display: Storlek i pixlar samt storlek i verkligheten (dvs. vad har den för förhållande mellan bredd:höjd)
10	Display: Implementera grundläggande färger
11	Display: Put/get pixel, grundläggande funktionalitet – vi KOMMER behöva återgå hit när vi vet hur gameboy:en faktiskt ritar på skärmen i detalj
12	Gränssnitt: MINIMALT fungerande gränssnitt för att rita ut skärmen för oss medans vi utvecklar
13	Ladda ROM från fil: läs in bytes och placera
14	ROM: Ska ha en sektion (bank) som alltid är läsbar, och X antal banker som programmet kan byta mellan och läsa
15	ROM: Fundera över hur detta implementeras – kommer förmodligen behöva återgå hit när vi har en bättre bild av hur det sker i praktiken, “vad programmen förväntar sig” utav emulatorn
16	Implementera instruktioner: Den här punkten är omöjlig att dela upp här och kommer ta den allra största delen av utvecklingen. Räkna med att det här steget egentligen tar upp säkert lika många steg till, som vi redan har. Ingen enskild lär ta överdrivet lång tid, men det finns många, och de ska fungera på väldigt specifika sätt.
17	Eventuellt: minnesinspektion; en lista (sökbar?) där vi kan slå upp olika minnesadresser och se vad som finns på dem.
18	Minnesinspektion: Implementera den logiskt. Hur ofta ska den uppdateras,

live eller vid paus av exekvering? (Det första kommer att ta prestanda – märkbart?)

19 Minnesinspektion: Visa upp minnesinspektionen i gränssnittet. Samma fönster som displayen, eller separat?

20 Dekompilering: Översätt alla maskininstruktioner till antingen det riktiga assemblyspråket för processorn (svårt) eller pseudo-assembly som ett proof-of-concept att visa upp.

21 Dekompilering: Visa dekompileringen i gränssnittet. Eventuellt i samma fönster som displayen.

22 Pie in the sky: implementera ljud, börja med hårdvaran

23 Ljud: Spela upp ljudet i gränssnittet.

...

4. Övriga implementationsförberedelser

I stort har detta redan beskrivits ovan, men ett generellt mål är att tänka objektorienterat: en emulator är ett program som inte “faller sig naturlig” att dela upp objektorienterat, så att fundera noga över lämpliga objektorienterade abstraktioner är något vi vill lägga fokus på.

I stort kommer vi ha en main-funktion som startar ett gränssnitt och själva emulator-klassen. Emulator-klassen kommer i sin tur ha fält för register, minne, display, etc, som är separata klasser. ROM-filen läser programmen av som vanliga minnesadresser, och därför kommer ROM-filen också vara ett fält som tillhör minnes-klassen.

5. Utveckling och samarbete

Vi har samma ambitionsnivå. Vi siktar båda på betyg 5, men har också valt ett svårt projekt med sikte på att lära oss kanske lite extra!

Vi är flexibla och jobbar ihop när tid finns och efter behov. Det bör nämnas att vi labbar ihop i 3 kurser, så *vad* vi jobbar med varje labbtillfälle bestäms oftast efter vad vi känner är mest brådskande för stunden. Vi har inga fasta “schemalagda” tider, utan bestämmer oftast när vi ska labba igen i slutet av varje pass. Vi jobbar under veckorna, men även på helger (lite efter behov). Resurstillfällena kommer vi förmodligen bara gå på om vi stöter på väldigt specifika problem, rent Java-mässiga eller relaterade till IntelliJ. (Vi vet inte hur mycket det är okej att fråga om på detaljnivå ang. själva emulatorn?)

Det är väldigt svårt att hitta en milstolpe som vi ska ha klarat vid ett visst datum, eftersom att implementationen av alla instruktioner (400-450 st) kommer att vara större delen av projektet (där vissa är extremt enkla, och andra är riktigt luriga).

Godtagbara skäl för att inte dyka upp på bestämda tider är oförutsedda hinder (sjukdom, dödsfall inom närmsta kretsen...) – så länge det inte är “jag orkade inte”, “jag

glömde bort det”, eller “jag glömde bort att jag behövde fixa något annat” så är vi okej med det.

Vi siktar på att alltid arbeta tillsammans, om vi inte har en *riktigt bra* anledning att göra ett undantag. Detta för att vi båda ska kunna ge input på vad vi skriver, och att båda ska hänga med i utvecklingsprocessen. Vi parprogrammerar alltid (dvs. tittar på, pratar om, och skriver) samma kodsnuitt. Vi har hittills alltid jobbat tillsammans.

Om Fritiof kallar sig själv gammal igen bryter vi dock **alla** labbgrupper.

(Resten av dokumentet ska inte lämnas in förrän projektet är klart, men **titta ändå genom allt för att se vilka delar ni behöver arbeta med och fylla i kontinuerligt under projektets gång!**)

Projektrapport

6. Implementationsbeskrivning

6.1 Milstolpar

1-22 är gjorda. Våra milstolpar gällande minnesinspektion och dekompilering utvecklades till en mer fullständig avlusare när vi väl skrev projektet, så de milstolparna ser små ut i planeringen, men kom att reflektera en större mängd arbete!

23 är ej implementerat. Ljud såg vi redan från början som den enklaste delen att undvika att implementera, medan man samtidigt fick någonting roligt att visa upp, och precis så blev det. Vi har inte ens haft tid att undersöka hur ljudet fungerar på Game Boy:en.

6.2 Dokumentation för programstruktur

Inledningsvis vill vi nämna att vi använt flera olika källor för att lära oss vad vi ska implementera, alltså hur en Game Boy fungerar. Att förklara programstrukturen är svårt utan att antingen räkna med att läsaren redan har koll på materialet, alternativt att skriva i sådan detalj att vi i princip skulle repetera, i detta dokument, all dokumentation vi läst. Vi har huvudsakligen använt [Pan Docs](#), som ger en utförlig bild av hur konsolen fungerar i sin helhet. Som komplement använde vi också [The Gameboy Emulator Development Guide](#), främst för att få en annan förklaring där Pan Docs kunde vara tvetydig när det kom till PPU:n. När vi behövde en mer lättöverskådlig och interaktiv vy av instruktionerna använde vi [Megan Sullivan's GB Opcodes](#), och slutligen [Rednex Game Boy Development System - CPU Instruction Documentation](#) för att få en andra förklaring på vissa instruktioner. Dessa källor har vi länkat till flitigt i våra Javadocs i källkoden.

Main-klassen är programmets utgångspunkt, och där programmets main-loop finns. Vi skapar där CPU- (inuti CPU-objektet skapas också ett Registers-objekt, för processorregistrena), PPU-, Memory- och Display-objekten, samt skapar objekten som hanterar användargränssnittet (EmuViewer, DebugViewer).

Eftersom att Java inte har stöd för några unsigned-siffror (exempelvis går en signed byte mellan -128 till +127, och en unsigned-byte går mellan 0 och 255) så behövde vi skriva våra egna klasser för funktionaliteten. Våra klasser (Unsigned (abstrakt) och UnsignedByte samt UnsignedShort (konkreta)) innehåller en vanlig integer (32-bitars signed heltal), som begränsas till de ovan nämnda värdena i deras konstruktorer eller när värdet ändras genom Unsigned.Set. Detta lägger grunden för att kunna hantera minnet i emulatorn som unsigned-tal mer eller mindre rakt igenom hela projektet. Värdena i minnet tolkas som signed endast i

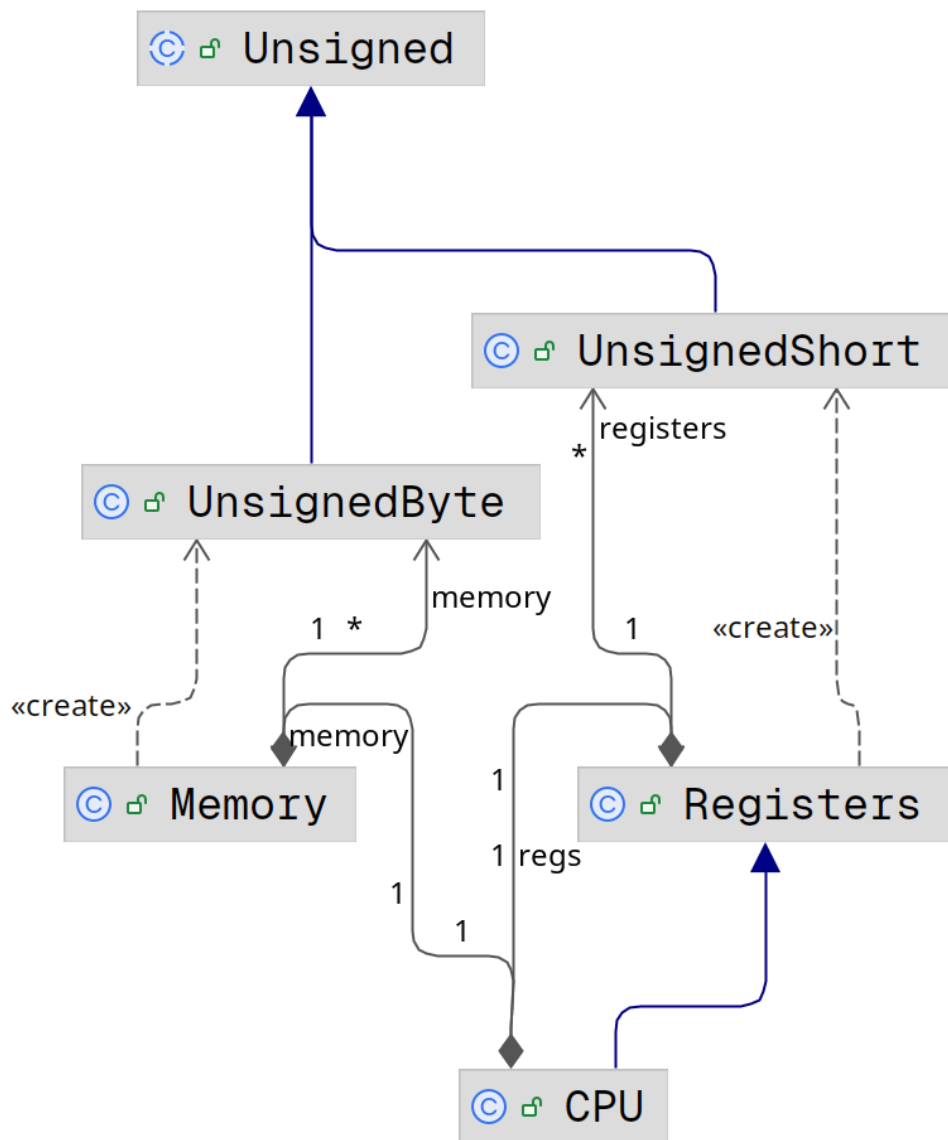
ett fåtal tillfällen, då de enkelt kan konverteras till vanliga bytes eller shorts.

Programmet börjar i ett tomt tillstånd, när inget spel är laddat. Genom metoder i gränssnittsklasserna (EmuViewer.changeROM) laddas en ROM-fil in, alternativt en tillståndsfil (genom EmuViewer.loadState). Fram tills att användaren gör det, står main-loopen stilla. Vi återgår till gränssnittsklasserna senare i dokumentationen.

När en ROM-fil är tillgänglig (efter att den har laddats in som ett ROM-objekt, vilket ligger inuti ett Memory-objekt) kan programmet börja exekvera ROM-filen, hädanefter refererad till som ett "spel" (trots att det i teorin kan vara ett godtyckligt program).

Main-loopen består av en yttre och en inre loop. Den yttre körs ovillkorligt, för att emulatorn ska spinna tomma cykler i väntan på att ett program laddas in. Den inre körs när ett program är tillgängligt, i samma antal klockcykler som Game Boy:en hann med på ca. 16 millisekunder. Detta för att vi då kör samma antal klockcykler som Game Boy:en hinner med på samma tid det tar den att rita ut en hel skärmbild, hädanefter refererad till som en "frame". Emulatorn kan då köra sina cykler, rita en frame, och vänta tills det är dags för nästa; resultatet blir att emulatorn låses till den hastighet Game Boy:en hade, och vi människor märker ingen skillnad trots att programmet effektivt inte gör någonting stora delar av tiden. Skulle programmet köras utan att artificiellt saktas ned skulle spelen gå alldeles för fort, och dessutom variera mellan olika datorer då det har olika prestanda.

CPU



När den inre loopen körs så består den av att CPU-objektet läser en instruktion från Memory-objektet, på samma adress (index) som programräknaren (som är ett av Game Boy:ens register, inuti Registers-klassen) för tillfället ligger. Kortfattat förklarat så läses en instruktion av, den avkodas, exekveras, och programräknaren ökar med korrekt antal (ty ett flertal av instruktionerna tar mer än en cykel att egentligen exekveras, även om vi gör allt på en gång i vår implementation).

I mer detalj, så läses CPU-instruktionerna in och identifieras via att först hämta instruktionen och sedan avkoda den fyra bitar i taget; först översta halvan av 8-bitars-instruktionen och sedan den understa. Genom denna uppdelning, första nibblen och andra nibblen, avgör vi vilken instruktion som ska exekveras genom en switch-sats, med $16 * 16 * 2$ möjligheter. Vid första anblick kan denna switch-sats anses som överväldigande stor, särskilt eftersom Gameboy-processorn har omkring 500 unika operationskoder. Men enligt vår bedömning är denna metod det tydligaste, mest strukturerade och mest effektiva sättet att hantera operationskoderna. För att korta ner på storleken av de

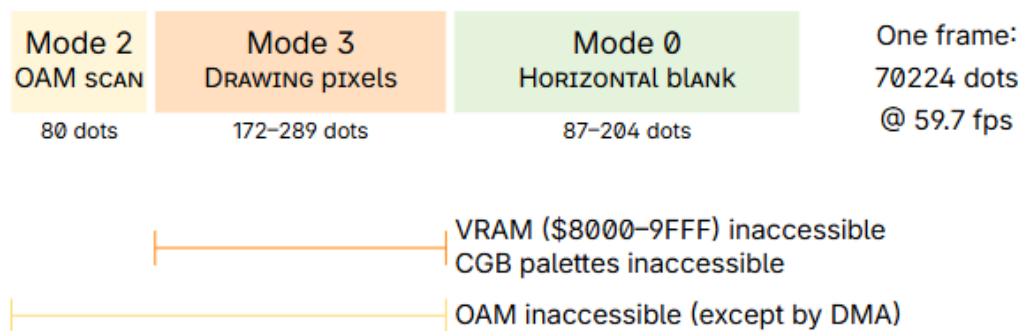
implementerade switchsatserna, både för 8-bitars och 16-bitars instruktionerna, letade vi efter mönster i instruktionstabellen, generaliserade gemensam logik och såg därmed till att implementationen undvek kod-duplicering, i den mån det gick. I den andra switchsatsen, som utgör 16-bitars instruktionerna, syns detta tydligt. Där ser man en tydlig representation på hur vi systematiskt har försökt generalisera instruktionslogiken för att motarbeta kod-duplicering och onödig komplexitet.

CPU-klassen har tillgång till Memory-objektet, och hanterar minnet genom Memory.read och Memory.write. Delar av minnet motsvarar hårdvaruregister och ROM-filen, varför Memory.read i många fall inte alls ger tillbaka någon siffra från den inre arrayen, utan använder specifik logik för den relevanta adressen. Det bör nämnas att det även finns Memory.unconditionalRead och Memory.unconditionalWrite, som PPU:n och diverse metoder i gränssnittsklasserna använder. Mer om detta under PPU-rubriken.

Registers-klassen ligger som fält inuti CPU-objektet, och innehåller en array av register, som utgörs av UnsignedShort-objekt. Beroende på instruktionen kommer registrena behandlas som 16-bitars-register eller som par av 8-bitars-register – exempelvis är register H de översta 8 bitarna av register HL, och L är de understa 8 bitarna. Vissa instruktioner kan ställa in värdet av H, L, eller av HL som helhet.

PPU

During a frame, the Game Boy's PPU cycles between four modes as follows:



1.1 Visuell representation av PPU cykeln (Pan Docs.)

PPU läge	Process
2	Läser in de objekt/sprites som överlappar den aktuella scanlinen
3	Skickar pixlar till LCD:n
0	Väntar till att nästa scanline ska börja
1 (Ej fig.)	VBlank, väntar på att nästa frame ska börja

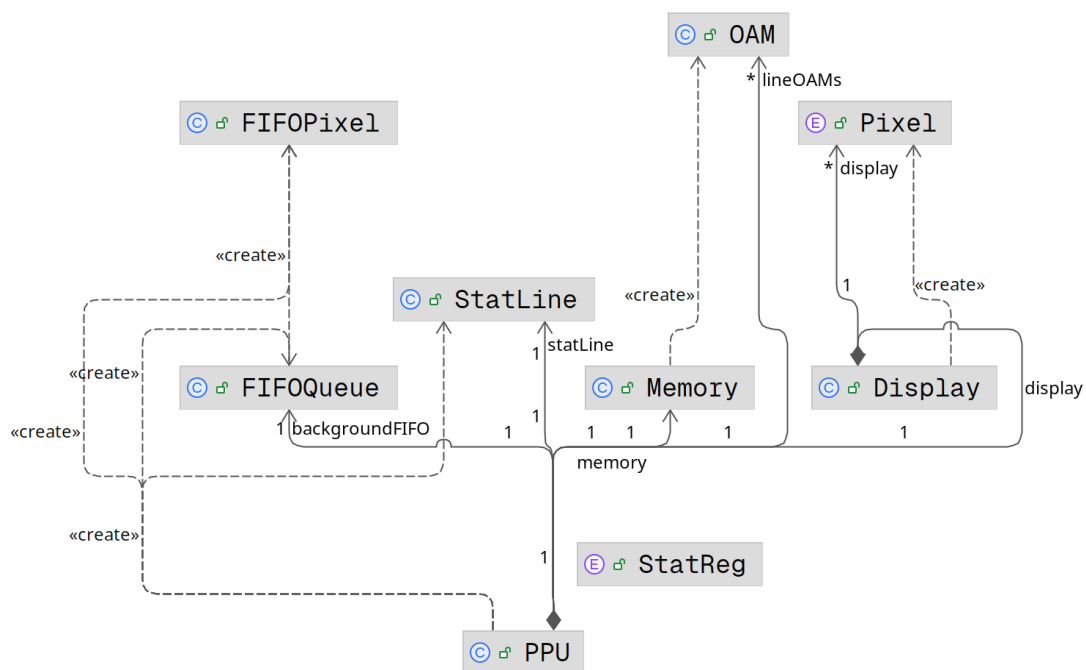
I samma loop kommer också PPU-klassen att kallas på. PPU:n (Picture Processing Unit) sköter grafiken i Game Boy:en, och vi har ju låst loopen till att köras så många gånger som konsolen ska hinna under målandet av en frame. Beroende på hur långt vi har kommit i en frame (vilken *scanline*, eller “horisontell rad”, av bilden vi kommit till, osv.) så gör PPU:n olika saker. Att förklara exakt hur PPU:n ska fungera är bortom detta dokumentets omfång, men kort förklarat har den 3 faser per scanline; den läser av OAM-minnet (Object Attribute Map, där grafiken för olika *objekt* eller *sprites* ligger, i programmet paketerade som OAM-objekt), den skriver pixlarna för en scanline till LCD-skärmen, och sedan “sover” den tills nästa scanline ska börja. Den 4:e fasen, *VBlank*, påbörjas efter att alla rader har ritats ut. VBlank utgör tiden mellan två frames, och är den huvudsakliga tiden som processorn kan komma åt VRAM (minnesadresserna \$8000 till \$9FFF) – denna hårdvarubegränsning är dock *inte* implementerad i vår emulator eftersom den kräver att emulatorn är perfekt synkroniserad med *timing*en av Game Boy:en. Eftersom att *vblank* är så viktig för programmen ska också en avbrottssignal skickas. PPU:n läser ur minnet med `Memory.unconditionalRead` och skriver med `Memory.unconditionalWrite`. Detta för att framtidssäkra programmet, om vi skulle implementera det korrekta beteendet att processorn periodvis inte kommer åt VRAM.

När PPU:n skriver pixlar till skärmen sker ett antal val mellan objektpixlarna (sprite) och bakgrundspixlarna, varför det finns två *köer* av pixlar (som *alltid* ska

innehålla 8 pixlar vardera) som PPU:n väljer mellan. Dessa var från början ArrayList-objekt, som var alldeles för långsamma, varför vår egen implementation av köerna består av bit-fält där 4 bitar motsvarar en pixel. PPU-objektet har alltså två instanser av FIFOQueue-objekt, som konverterar mellan bitar och FIFOPixel-objekt (en väldigt liten, simpel klass som paketerar ihop en pixels egenskaper).

Displayen är en tvådimensionell array av 160*144 Pixel-värden. Pixel är en enumeration av fyra olika värden; Game Boy:en kunde visa 4 olika färger (egentligen nyanser av samma färg, på original Game Boy:en grön) på sin display, och vi behöver därför bara avgöra vilken nyans som ska vara var på den interna skärmen, innan vi till sist avgör hur en sådan pixel ska se ut i vårt användargränssnitt. Mer om gränssnittet senare i dokumentet.

Den sista delen av PPU:n är StatLine klassen, som är svårförklarad utan att djupa sig i Game Boy:ens interna detaljer. Enkelt förklarat är det en av-på signal som kan slås på av flera olika anledningar (exakt vilka beror på, och ställs in av, spelet), och en *stigande kant* på signalen ska skicka en avbrottssignal. Om signalen går från av till på skickas alltså en avbrottssignal, men om signalen fortsätter vara på nästa gång något läggs till så skickas alltså *inte* ett till avbrott. Därför bor dessa egenskaper i sin egna klass som hanterar denna detalj.

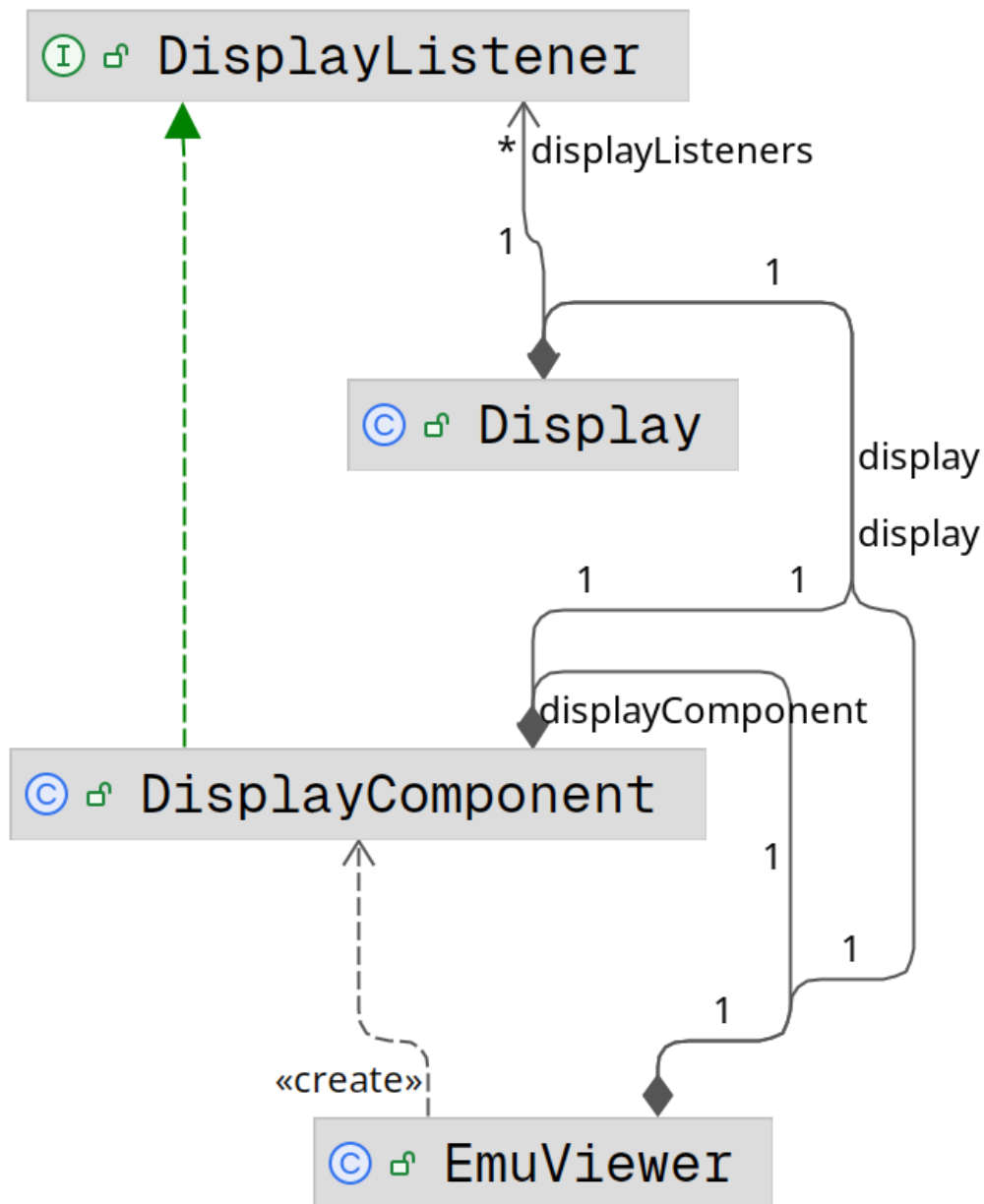


Till sist finns också hårdvarutimers i Game Boy:en. De uppdateras i main-loopen efter att CPU:n har läst och exekverat en instruktion. Eftersom att instruktionen kan ta olika mängder av *egentliga* klockcykler kommer också våra timers uppdateras beroende på hur många det tog. Dessa timers har diverse inställningar som spelet ställer in efter behov, och kan också skicka en avbrottssignal när ena timern överflödar.

När loopen för en frame är klar så påbörjas alltså en ny. CPU:n fortsätter som

vanligt, och PPU:n kommer börja om på första raden av en ny skärmbild. Denna loop är kärnan i emulatorn.

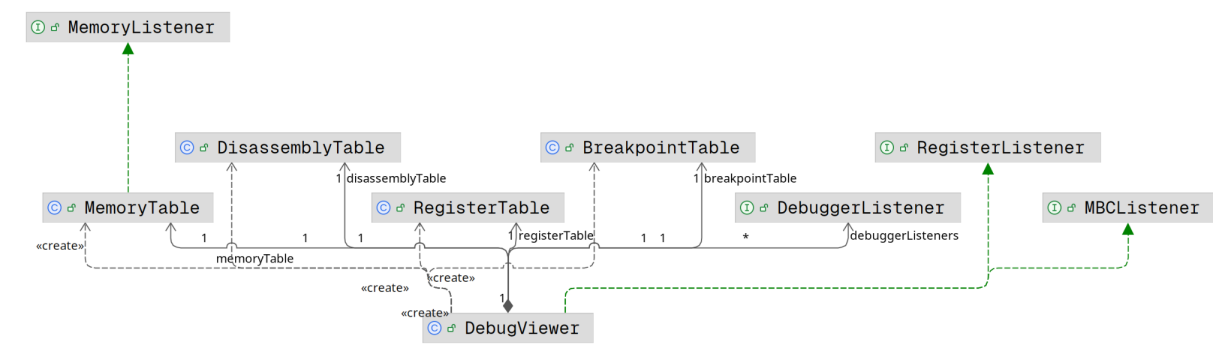
Grafiska gränssnittet



EmuViewer är klassen som hanterar det huvudsakliga grafiska gränssnittet. Den har en komponent, **DisplayComponent**, som översätter vad som finns i **Display**-objektet (den *interna* skärmen) till faktiska färger, och målar ut dem i gränssnittet. Den skalar också upp grafiken till att passa fönsterstorleken, som avgörs av användaren. Komponenten är en **DisplayListener**, och **Display**-objektet kallar på sina lyssnare när en skärmbild har målats färdigt.

EmuViewer lyssnar också efter knapptryck, och visar upp en **PopupMenu** när användaren högerklickar på skärmen. Många av knapptrycken reflekterar knapparna på Game Boy:en, och ställer därför in värden i **Memory**-objektet, som EmuViewer har en referens till. Dessa värden ska ju kunna läsas av av spelet, och knapptryck ska eventuellt kunna skicka avbrottssignaler, om spelet ställt in detta.

Avlusaren



Delarna av avlusaren är en dekompileringsvy (visar alla instruktioner i ROM-filen översatta till assembly-instruktioner), en registervy (visar värdena i alla register), en brytpunktsvy (en lista på de brytpunkter som skapats) och en vy för minnet (lista av alla värden i minnet). Själva implementationerna är väldigt enkla, där for-loops fyller våra JTables med värden, och, där det går, uppdateras individuellt allt eftersom värdena uppdateras i emulatorn. När MBC-objektet byter ROM-bank uppdateras dock *hela* dekompileringsvyn för att reflektera bankbytet.

När man pausar eller stegar genom programmet belyses den nuvarande instruktionen, för att användaren lätt ska kunna följa var programmet befinner sig.

14

användaren själv fyller genom funktioner i användargränssnittet (genom att välja det i panelen i avlusaren, eller trycka på snabbkommandot).

MBC

I ROM-objektet laddas alla bytes från en binär ROM-fil till en array. Utan att gå in i Game Boy:ens tekniska detaljer för mycket så kan ett spel endast adressera \$8000 bytes i ROM-filen utan någon hjälp, alltså 32 KiB. Många spel är mycket större än så (de största är 1-2 MiB), och för att spelet ska kunna adressera allt minne i kassetten används en Memory Bank Controller.

De enklaste spelen som Tetris eller Dr. Mario ryms i 32 KiB och saknar en MBC. Större spel har någon form av MBC (det finns MBC1, 2, ..., 5, och ett par andra varianter). Spelet "skriver" sedan till ROM-adresserna för att kontrollera MBC-chippet.

Det här har vi implementerat som en abstrakt MBC-klass, med konkreta implementationer som väljs utifrån information i kassett-headern (en del av det tidiga minnet i ROM-filerna innehåller meta-information om ROM:en) som en del av ROM-objektets konstruktor. Alla läsningar av ROM-objektet dirigeras genom MBC-objektet, och kan peka på olika "banker" av ROM-arrayen.

Samma princip gäller också RAM-minnet, om sådant finns, på kassetten. RAM-minnet på kassetten används ofta tillsammans med ett batteri, som alltså höll RAM-minnet vid liv mellan att Game Boy:en var påslagen, och kunde därför innehålla sparningsdata mellan speltillfällena.

Serialisering

Flera delar av emulatorn kan serialiseras. Ovan nämndes att RAM-minnet i kassetten kunde innehålla sparningsdata, och vår emulator måste alltså kunna bibehålla RAM-minnet mellan speltillfällen precis som en verklig kassett kunde göra det. RAM-arrayen serialiseras m.h.a. Gson-biblioteket, och sparas till en json fil med samma namn som ROM-filen i samband med att emulatorn stängs ned, med en annan filändelse. Om en matchande .sav-fil finns när ett spel laddas, som har meta-informationen att ett batteri finns i kassetten, så deserialiseras i stället sparningsfilen och laddas in i RAM-arrayen. Skulle en sådan fil saknas initialiseras RAM-arrayen i stället till basvärden (nollor).

Vidare erbjuder vi funktionaliteten att spara hela emulatorns tillstånd till json fil, en tillståndsfil (engelska: save state), vilket är en vanlig funktion i emulatorer. CPU-objektet, Registers-objektet, Memory-objektet, ROM-objektet och MBC-objektet samlas ihop i ett wrapper-objekt som i sin tur serialiseras. Vi stötte här på problem eftersom att Gson krävde typinformation för att kunna deserialisera det konkreta MBC-objektet till det abstrakta fältet i ROM-objektet. Det fanns flera lösningsalternativ, men eftersom att vårt problem var isolerat till MBC-klasserna implementerades ett s.k. Data Transfer Object, SerializableMBC; varje MBC kan skapa en SerializableMBC utifrån sig själv, där typinformationen

bibehålls, och varje SerializableMBC kan skapa ett konkret MBC-objekt utifrån sig själv. På detta sätt undviker vi problemet, och kan serialisera och återställa emulatorns tillstånd utifrån fil, vilket alltså i praktiken låter användaren spara godtyckliga ögonblick i spelen till filer, och inte endast då spelen låter användaren spara (om spelet tillåter det alls).

Abstraktioner

I vissa fall håller sig koden i programmet mindre abstraherad, exempelvis undviks Map-objekt och switch-satser används i stället. Detta var ett resultat av att Map-objekten var väldigt långsamma, jämförelsevis. Till viss del kan det handla om för tidig optimering (engelska: *premature optimization*).

Men vid profilering av programmet finner vi att många abstraktioner (speciellt Map-objekt) är, utan överdrift, hundratals gånger långsammare än switch-case motsatsen (troligtvis eftersom kompilatorn kan optimera switch-satser på ett mycket effektivare sätt). Vi vill därför påtala att det i många fall är medvetna val att hålla programmet mindre abstrakt, på vissa håll. Visst förbehåll lämnas dock för okunskap.

Kortfattat; fördelarna med Maps saknas i vårt program, medan alla nackdelarna syns väldigt tydligt, och därför anses de olämpliga. Då vi vill kunna utöka programmet i framtiden på sätt vi inte ännu vet hur kostsamma de kommer att bli, vill vi varken ägna oss åt förtida optimeringar, men inte heller onödiga abstraktioner som inte för med sig tydliga fördelar.

Loggaren

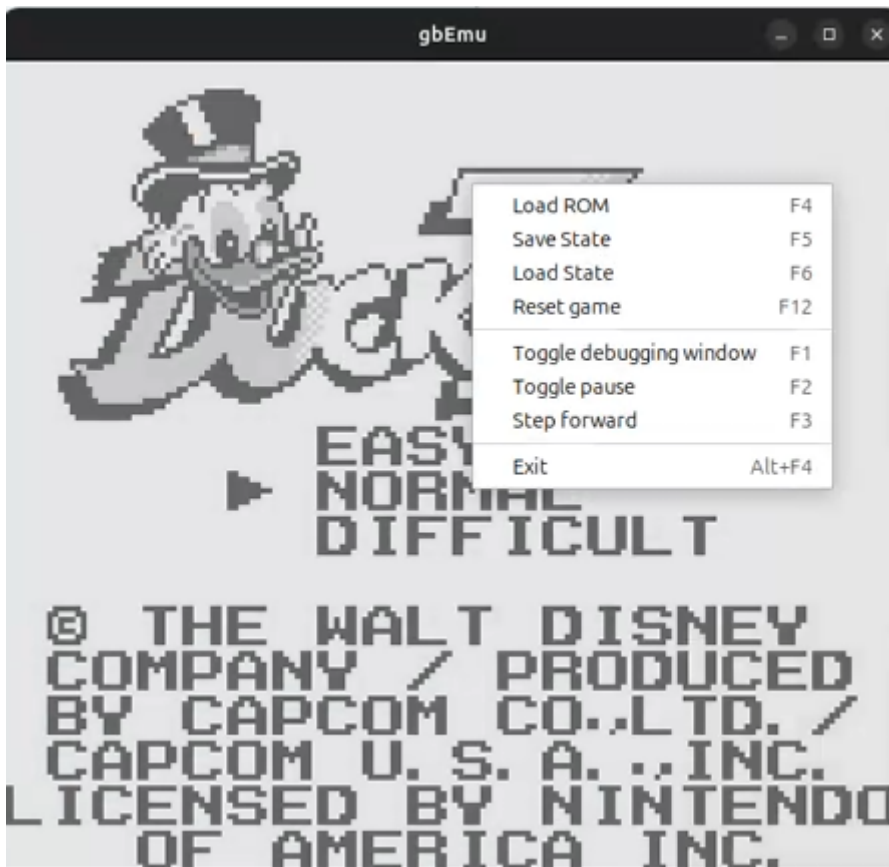
Vid olika tillfällen, då det uppstår något typ av error, ett exception fångas, eller när vi vill spara undan information (till exempel hur emulatorn tolkar headern av en ROM-fil) så loggas händelsen till en loggfil med hjälp av en `java.util.logging.Logger`, i `CuteLogger`-klassen. Det är en väldigt grundläggande implementering, som används på flera ställen i projektet. I stort kan emulatorn graciöst hantera eventuella fel som kan uppstå, men ett par fel bedöms vara omöjliga att återhämta sig från, delvis om spelen skulle skriva orimliga saker till speciella minnesadresser. Det skulle inte fungera på den riktiga hårdvaran, så vad som ska hända i emulatorn är inte heller definierat, varför vi väljer att avbryta helt när vi stöter på vissa problem.

7. Manual



gbEmu är en Game Boy-emulator, som alltså låter en ladda Game Boy-spel (lättast med filändelsen ".gb") och spela dem. Även avlusningsverktyg finns i programmet, som används för att utveckla programmet men också kan vara intressant för den nyfikne, eller den som skrivit sitt egna Game Boy-spel.

När du öppnar programmet är inget spel laddat. En uppsättning testprogram som demonstrerar att processorinstruktionerna fungerar följer med, men för att prova att spela något spel måste användaren tillhandahålla en ROM-fil själv (av juridiska skäl).



För att ladda en ROM-fil högerklickar man på fönstret och väljer "Load ROM", eller trycker på F4-knappen på tangentbordet. I högerklicksmenyn finns även andra alternativ, som att spara eller ladda emulatorns tillstånd till fil, stega genom programmet, eller öppna det utökade avlusningsfönstret. Obs: Om tillståndet sparas till fil är det lättast att ge filen ändelsen ".state", för att underlätta att hitta dem. Du behöver inte först ladda en ROM-fil om du ämnar ladda en tillståndsfil, och tillståndsfilen behöver inte heller matcha inladdad ROM-fil, om en finns. Du kan ändra fönsterstorlek efter behov, och grafiken kommer skalas för att passa den nya fönsterstorleken.

Debugger

File

Debug

Address	Instruction	Register	Value
\$0000	LD	A	\$10
\$0009	NOP	F	\$00
\$000A	NOP	B	\$01
\$000B	NOP	C	\$00
\$000C	NOP	D	\$10
\$000D	NOP	E	\$47
\$000E	NOP	H	\$71
\$000F	NOP	L	\$10
\$0010	87	ADD A, A	\$DFF3
\$0011	E1	POP HL	\$0025
\$0012	D5	PUSH, DE	
\$0013	18	JR \$0002	
\$0015	00	NOP	
\$0016	00	NOP	
\$0017	00	NOP	
\$0018	85	ADD A, L	
\$0019	6F	LD L, A	
\$001A	D0	RET NC	
\$001B	24	INC H	
\$001C	C9	RET	
\$001D	00	NOP	
\$001E	00	NOP	
\$001F	00	NOP	
\$0020	85	ADD A, L	
\$0021	6F	LD L, A	
\$0022	30	JR NC, \$0025	
\$0024	24	INC H	
\$0025	7E	LD A, (HL)	

Breakpoint

Address	\$0	\$1	\$2	\$3	\$4	\$5	\$6	\$7	\$8	\$9	\$A	\$B	\$C	\$D	\$E	\$F
\$0000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$00F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$0190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$01A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
\$01B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Avlusningsfönstret består av:

1. En disassembly vyn av ROM-filen och resten av minnet, men observera att majoriteten av minnet förbi adressen \$7FFF inte tillhör ROM-filen och i stället är arbetsminne. Med andra ord är detta en vy av instruktionerna från programfilen översatta till den motsvarande assembly-instruktionen. När man pausar emulatorn, eller stegar genom programmet, markeras instruktionen som emulatorn befinner sig på.
2. En vy av alla processorns register och vad som finns i dem. Dessa uppdateras endast när emulatorn är pausad.
3. En vy av alla brytpunkter, som man kan lägga till brytpunkter i genom panelen i avlusningsfönstret (under "Debug") alternativt genom att trycka på F9 (lägg till brytpunkt) och F10 (ta bort brytpunkt). Under "Debug" kan man även ta bort samtliga skapade brytpunkter på en gång. När brytpunkter är satta kommer emulatorn automatiskt pausas när programräknaren når brytpunktens adress.
4. En vy av minnet, och vad som finns i varje adress. Observera att adresserna \$0000 till \$7FFF tillhör ROM-filen, och därför oftast är mer lämpliga att läsas av som instruktioner i disassembly vyn snarare än numeriska värden i minnesvyn. Observera att gbEmu inte ännu stödjer ljud! Det saknas ävem stöd för spel som använder någon annan memory bank controller än MBC1, alternativt ingen alls. Spel som kan spara går bra att spela, men själva sparandet fungerar inte. Om man vill spara spelet kan man istället spara emulatorns tillstånd till fil. I slutet av dokumentationen finns en lista på bekräftat fungerande spel.

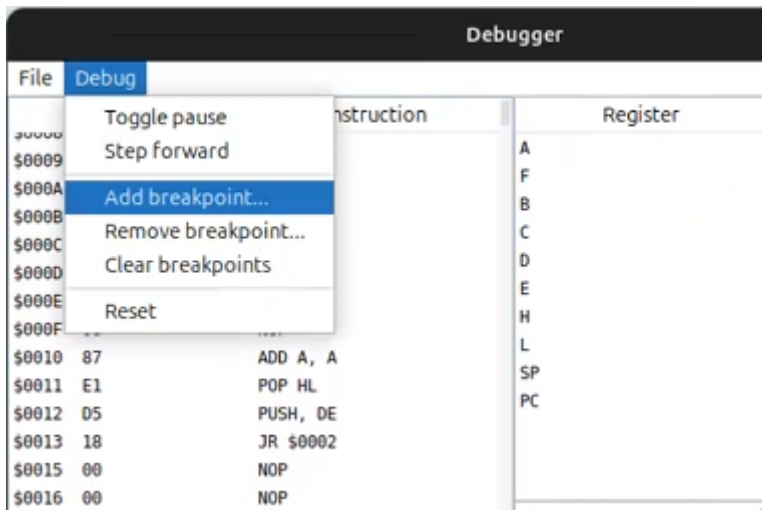
Kontroller för spel:

- WASD: pil-knapparna
- O: A-knappen
- K: B-knappen
- T: Start-knappen
- Y: Select-knappen

Kortkommandon:

- F1: Öppna avlusningsfönstret (och pausa emulatorn)
- F2: Pausa/fortsätt emulation
- F3: Ta ett steg framåt i emulationen (pausar även emulatorn)
- F4: Ladda in (eller byt) ROM-fil
- F5: Spara tillstånd till fil
- F6: Ladda tillstånd från fil
- F9: Lägg till brytpunkt
- F10: Ta bort brytpunkt
- F12: Starta om (och pausa) nuvarande spel

Ett urval av dessa (de relevanta för att spela spel) finns i högerklicksmenyn i spelfönstret, men allihopa finns i avlusningsfönstrets panelmeny. Avlusningsfönstret stödjer endast kortkommandona F2 och F3 för att pausa/fortsätta respektive stega framåt.



Bekräftat fungerande spel

- Kirby's Dream Land
- Kirby's Dream Land 2
- Super Mario Land
- Zelda: Link's Awakening
- Tetris
- Bionic Commando
- Castlevania II: Belmont's Revenge
- Dr. Mario
- Metroid II: Return of Samus
- Catrap

Det är dock troligt att många andra också fungerar!